codingninjas

Class 7: Advanced Kubernetes

Session Overview

- **Kubernetes Networking**: Understand Kubernetes' networking model, Services, and Ingress.
- **Kubernetes Storage**: Learn about Persistent Volumes (PV), Persistent Volume Claims (PVC), and Storage Classes in Kubernetes.
- **Scaling Applications**: Explore horizontal and vertical scaling of applications in Kubernetes, and configure auto-scaling based on resource utilization.

Prerequisites

- Basic understanding of containerization (Docker) and Kubernetes basics.
- Familiarity with YAML files and Kubernetes CLI commands is recommended.

Introduction

In this session, we'll dive deeper into advanced Kubernetes concepts that are essential for managing production-grade applications. You will learn how Kubernetes handles networking, storage, monitoring, and scaling in a highly dynamic environment. We will explore how to expose your applications to the outside world using Services and Ingress, configure persistent storage, monitor your clusters, and auto-scale applications to meet demand. This class will focus on hands-on examples to give you practical experience with these advanced Kubernetes features, preparing you for real-world challenges.

Kubernetes Networking

Kubernetes networking is built to support the dynamic, distributed nature of containerized applications. Each pod in a Kubernetes cluster is assigned a unique IP address, and all containers within the pod share that IP address. Kubernetes provides several networking resources to enable communication within the cluster and with the outside world, including **Services**, **Ingress**, and **Network Policies**.

Key Components of Kubernetes Networking:

1. **Pods:** The smallest unit in Kubernetes, each pod gets its own IP address.
2. **Services:** Services enable communication between pods and ensure a stable endpoint for other services or external clients, even when pod IPs change.
3. **Ingress:** Ingress allows external access to services in the cluster, typically via HTTP/HTTPS, and supports load balancing, SSL termination, and name-based virtual hosting.

Types of Services:

- **ClusterIP:** Exposes the service within the cluster (default).
- **NodePort:** Exposes the service on a specific port on each node in the cluster.
- **LoadBalancer:** Exposes the service externally using a cloud provider's load balancer.
- **ExternalName:** Maps the service to an external hostname.

Hands-On Lab: Exposing Applications Using Services and Ingress

Objective:
In this lab, you'll deploy a sample application and expose it externally using Services and Ingress.

Lab Setup:

Prerequisites:

- A Kubernetes cluster (Minikube or any cloud provider setup).
- `kubectl` command-line tool configured to interact with your Kubernetes cluster.
- Basic understanding of Docker and containerized applications.

Step 1: Deploying a Simple Application Deploy a simple Nginx application on your Kubernetes cluster.

```
kubectl create deployment nginx-app --image=nginx
```

Check if the deployment is successful:

```
kubectl get deployments
```

Step 2: Exposing the Application Using a Service Expose the application using a **NodePort** service, which will allow access to the application from outside the cluster.

```
kubectl expose deployment nginx-app --type=NodePort --port=80
```

Verify the service:

```
kubectl get services
```

You will see the NodePort and its corresponding external IP for access.

Step 3: Configuring Ingress To expose the application via Ingress (with a friendly URL), create an Ingress resource.

First, create a YAML file (`ingress.yaml`) for the Ingress resource:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: nginx-ingress
spec:
  rules:
  - host: nginx.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: nginx-app
            port:
              number: 80
```

Apply the Ingress resource:

```
kubectl apply -f ingress.yaml
```

Make sure that Ingress controllers (like NGINX Ingress Controller) are installed and running in your cluster.

Step 4: Access the Application Once the Ingress is set up, you can access your application via `nginx.example.com` (you may need to add this entry to your `/etc/hosts` file if running locally).

Kubernetes Storage

Kubernetes provides a flexible and powerful model for managing storage for containerized applications. Kubernetes decouples the storage from the compute nodes, allowing pods to be stateless while enabling persistent data storage across different clusters and nodes.

Kubernetes abstracts storage into different components, namely **Persistent Volumes (PVs)**, **Persistent Volume Claims (PVCs)**, and **Storage Classes**.

Key Kubernetes Storage Components:

- **Persistent Volumes (PV):** A piece of storage in the cluster that has been provisioned by an administrator or dynamically through a storage class.
- **Persistent Volume Claims (PVC):** A request for storage by a user. It binds a PV to a pod.
- **Storage Classes:** Provide a way to define different classes of storage (e.g., SSDs, networked storage) and allow dynamic provisioning of PVs based on the storage class definitions.

Hands-On Lab: Configuring Persistent Storage in Kubernetes

Objective:
In this lab, you'll learn how to configure persistent storage for a pod using Persistent Volumes (PVs), Persistent Volume Claims (PVCs), and Storage Classes.

Lab Setup:

1. **Prerequisites:**
    - A Kubernetes cluster (Minikube or any cloud provider setup).
    - `kubectl` command-line tool configured to interact with your Kubernetes cluster.
    - Basic understanding of Docker and Kubernetes concepts.

Step 1: Setting Up a Persistent Volume (PV)

A **Persistent Volume** is a cluster resource, much like a node, that exists independently of any individual pod. You can either provision the PV statically or dynamically based on Storage Classes.

Create a **Persistent Volume** YAML file (`pv.yaml`) as follows:

```
apiVersion: v1
kind: PersistentVolume
metadata:
```

```
  name: my-pv
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: manual
  hostPath:
    path: "/mnt/data"
```

This defines a PV with 1Gi of storage, which can be read/write by only one pod at a time. The storage is backed by the host node directory `/mnt/data`.

Apply the PV to your Kubernetes cluster:

```
kubectl apply -f pv.yaml
```

Check the status of the PV:

```
kubectl get pv
```

Step 2: Creating a Persistent Volume Claim (PVC)

A **Persistent Volume Claim (PVC)** is a request for storage by a pod. You specify the size and access mode (e.g., read-write, read-only), and Kubernetes binds a matching PV to the PVC.

Create a **PVC** YAML file (`pvc.yaml`):

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: my-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
```

```
    requests:
      storage: 1Gi
  storageClassName: manual
```

This requests a 1Gi volume with the same access mode as the PV (`ReadWriteOnce`).

Apply the PVC to your Kubernetes cluster:

```
kubectl apply -f pvc.yaml
```

Verify that the PVC has been successfully created and bound to a PV:

```
kubectl get pvc
```

Step 3: Attaching the PVC to a Pod

Now that you have the PV and PVC ready, you can attach the PVC to a pod. The PVC will ensure that the pod can access the persistent storage.

Create a **Pod** YAML file (`pod.yaml`) that uses the PVC for storage:

```
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  containers:
  - name: my-container
    image: nginx
    volumeMounts:
    - mountPath: "/usr/share/nginx/html"
      name: my-storage
```

```
  volumes:
  - name: my-storage
    persistentVolumeClaim:
      claimName: my-pvc
```

In this configuration, the pod will mount the PVC at `/usr/share/nginx/html`, meaning any data saved there will persist across pod restarts.

Apply the pod configuration:

```
kubectl apply -f pod.yaml
```

Check the status of the pod:

```
kubectl get pods
```

Step 4: Verifying the Storage

You can verify that the pod is using the persistent storage by logging into the pod and creating files inside the mounted volume.

First, access the running pod:

```
kubectl exec -it my-pod -- /bin/bash
```

Then, navigate to the mount path and create a file:

```
cd /usr/share/nginx/html
echo "Hello, Kubernetes Storage!" > index.html
```

Exit the pod and restart it:

```
kubectl delete pod my-pod
kubectl apply -f pod.yaml
```

After the pod restarts, you can re-enter the pod and verify that the file still exists, confirming that the data is stored persistently:

```
kubectl exec -it my-pod -- /bin/bash
cat /usr/share/nginx/html/index.html
```

Step 5: Using Dynamic Provisioning with Storage Classes

While the above example demonstrates static provisioning, Kubernetes also supports **dynamic provisioning** with **Storage Classes**. In this setup, you don't have to manually create PVs; they are automatically created based on the storage class definition.

Create a **Storage Class** YAML file (`storageclass.yaml`):

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-storage
provisioner: kubernetes.io/no-provisioner
volumeBindingMode: Immediate
```

Then modify your **PVC** to reference the new storage class:

```
apiVersion: v1
kind: PersistentVolumeClaim
```

```
metadata:
  name: dynamic-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
  storageClassName: fast-storage
```

With this setup, Kubernetes will dynamically create a PV using the defined storage class.

Hands-On Lab: Scaling Applications in Kubernetes

Objective: In this hands-on lab, you will deploy a sample application and configure both horizontal and vertical scaling for it. You will use HPA to automatically scale your application based on CPU usage.

Lab Setup:

Prerequisites:

- A running Kubernetes cluster (Minikube or any cloud provider).
- `kubectl` command-line tool configured to interact with your cluster.
- Basic understanding of Kubernetes deployments and services.

Step 1: Deploying a Sample Application

Create a Deployment for a Sample Application:
We'll use a simple web application that simulates high CPU usage. Create a deployment YAML file named `nginx-deployment.yaml`.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
```

```
    app: nginx
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        resources:
          requests:
            cpu: "200m"
            memory: "512Mi"
          limits:
            cpu: "500m"
            memory: "1Gi"
        ports:
        - containerPort: 80
```

Deploy the Application:

Run the following command to create the deployment:

```
kubectl apply -f nginx-deployment.yaml
```

Verify the Deployment:

Check if the deployment was created successfully:

```
kubectl get deployments
```

1. You should see the `nginx-deployment` listed with 2 replicas.

Step 2: Creating a Service to Expose the Application

1. **Create a Service YAML file named `nginx-service.yaml`:**

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  type: LoadBalancer
  ports:
    - port: 80
      targetPort: 80
  selector:
    app: nginx
```

Apply the Service Configuration:

```
kubectl apply -f nginx-service.yaml
```

Verify the Service:

Check if the service was created successfully:

```
kubectl get services
```

Note the `EXTERNAL-IP` (if applicable) to access your application in a web browser.

Step 3: Testing Application Load

To test the autoscaling feature, we need to generate CPU load on our application. You can use a simple script to send requests to the nginx server.

1. **Run a Load Testing Tool:**
   You can use a load testing tool like `hey` or `ab` (Apache Benchmark) to generate load. If you don't have these tools installed, you can use a temporary pod with `busybox`.

```
kubectl run load-generator --image=busybox /bin/sh -c "while true; do wget -q -O- http://nginx-service; done"
```

This command will continuously send requests to the nginx service, causing CPU load.

Step 4: Configuring Horizontal Pod Autoscaler (HPA)

Create the Horizontal Pod Autoscaler:
Use the following command to create an HPA that targets the nginx deployment. The HPA will scale the deployment based on CPU utilization, maintaining a target of 50%.

```
kubectl autoscale deployment nginx-deployment --cpu-percent=50 --min=2 --max=10
```

Verify the HPA:

Check the HPA status with the following command:

```
kubectl get hpa
```

You should see the `nginx-deployment` HPA with the current CPU utilization and number of replicas.

Step 5: Monitoring Scaling Behavior

Monitor the Scaling:
As the load generator continues to run and the CPU usage increases, Kubernetes will automatically scale the number of replicas in the `nginx-deployment` based on the defined

HPA.
You can monitor the status of the deployment and see the number of replicas change with:

```
kubectl get deployments
```

Stop the Load Generator:

Once you see that the autoscaler is functioning correctly, you can stop the load generator pod:

```
kubectl delete pod load-generator
```

Step 6: Vertical Scaling of the Application

Edit the Deployment to Change Resource Requests and Limits:
Update the `nginx-deployment.yaml` file to increase the CPU and memory resources. For example, change it to:

```
resources:
  requests:
    cpu: "500m"
    memory: "1Gi"
  limits:
    cpu: "1"
    memory: "2Gi"
```

Apply the Changes:

Run the following command to update the deployment:

```
kubectl apply -f nginx-deployment.yaml
```

Verify the Changes:

Check the updated resource allocations:

```
kubectl get deployment nginx-deployment -o yaml
```